Solving Small Linear Systems with the HHL Algorithm: A Hands-On Tutorial in Cirq and Qiskit

Course number and title, Fall 2026

YOUR NAME, Your University, USA

The Harrow–Hassidim–Lloyd (HHL) algorithm prepares a quantum state whose amplitudes are proportional to the solution of a linear system $Ax = b$. This paper is a hands-on tutorial that builds the algorithm twice, in Cirq and in Qiskit, and runs it on four small systems taken from the statics of spring-coupled blocks: a diagonal 2×2 system, a coupled 2×2 system whose eigenvalues are never used, a 4×4 system, and a 3×3 system padded to 4×4. The Cirq version starts from a textbook listing that no longer runs on current Cirq; we document the repairs and show that one of them, an evolution time that is never handed to the solver, explains a mismatch in the textbook's printed output. The two free parameters of the algorithm are set from Gershgorin bounds instead of eigenvalues. On a noiseless simulator the two implementations agree to machine precision, and with six clock qubits and 8192 shots the sampled solutions lie within 3.3 % of the classical ones in Euclidean norm. Each additional clock qubit roughly halves the algorithmic error and multiplies the number of CNOT gates by 1.5 to 1.6. Throughout, we spell out how A and b enter the circuit and how measurement counts are turned back into numbers.

CCS Concepts: • **Theory of computation** → **Quantum computation theory**; • **Computer systems organization** → **Quantum computing**; • **Mathematics of computing** → *Solvers*.

Additional Key Words and Phrases: quantum linear systems, HHL algorithm, quantum phase estimation, amplitude encoding, Cirq, Qiskit

1 INTRODUCTION

Solving a system of linear equations $Ax = b$ is the workhorse of scientific computing: finite-element models, circuit simulators and least-squares fits all reduce to it. Classical solvers need time that grows at least linearly with the number of unknowns N. In 2009 Harrow, Hassidim and Lloyd (HHL) showed that, under certain conditions, a quantum computer can do the job in time polynomial in $\log N$ and in the condition number κ of A, which is an exponential improvement in N over the best classical algorithms [12].

The conditions matter [1]. The vector b has to be loaded into a quantum state efficiently, the operator e^{iAt} has to be implemented efficiently, κ must not be large, and the answer comes out as a quantum state $|x\rangle$ instead of a list of numbers: each run of the circuit yields a single sample, and learning all N entries of x takes many runs. HHL is therefore best seen as a subroutine that hands $|x\rangle$ to further quantum processing, and as the standard example of how phase estimation can be turned into matrix arithmetic [11].

This paper is a tutorial. Reading about HHL is easier than implementing it. The algorithm chains together state preparation, Hamiltonian evolution, phase estimation, a controlled rotation, uncomputation and post-selection, and every link has a bit order, a sign or a scaling constant that has to match its neighbours. Our goal is to make each of these choices explicit, on systems small enough to check by hand, and in two frameworks so that one implementation can be tested against the other. The paper contributes the following.

- A working Cirq implementation, obtained by repairing and extending the HHL listing of Pattanayak's textbook [19]. Every change is documented (Section 4.1), including the diagnosis of a mismatch between the numbers the listing expects and the numbers the book prints (Section 5.2).

Author's address: Your Name, your.email@university.edu, Your University, City, State, USA.

- An independent Qiskit implementation written from scratch (Section 4.2). The two programs agree to within 1.2×10^{-14} on every exact output probability.
- A recipe for choosing the two free parameters of HHL, the evolution time t and the rotation constant C, from cheap eigenvalue bounds instead of the eigenvalues themselves (Section 3.4).
- An explicit decoding rule that turns measurement counts into the entries of x with error bars, including the norm $\|x\|$ that the normalised quantum state seems to have lost (Section 4.3).
- Experiments on four linear systems up to 4×4, and a study of how accuracy and gate count change with the size of the clock register (Section 5).

Section 2 reviews the algorithm, Section 3 explains how a concrete linear system is encoded, Section 4 describes the two programs, and Section 6 collects the lessons we would pass on to the next student.

2 BACKGROUND

2.1 Qubits, amplitudes and measurement

A register of q qubits is described by 2^q complex numbers, the *amplitudes*, one for each bit string of length q. We write the state as $\sum_i a_i |i\rangle$ with $\sum_i |a_i|^2 = 1$, where $|i\rangle$ stands for the i-th bit string in counting order ($0\ldots00$, $0\ldots01$, and so on). Gates are unitary matrices acting on the vector of amplitudes. A measurement returns bit string i with probability $|a_i|^2$ and nothing else; in particular it never shows an amplitude directly. Two consequences shape everything below: a vector of N numbers fits into only $\log_2 N$ qubits, and the only way to learn about those numbers is to repeat the experiment and count outcomes [18].

2.2 What "solving" means on a quantum computer

Let A be an $N \times N$ Hermitian matrix with $N = 2^m$. It has real eigenvalues λ_j and orthonormal eigenvectors $|u_j\rangle$, so $A = \sum_j \lambda_j |u_j\rangle\langle u_j|$. Expanding the normalised right-hand side in the same basis, $|b\rangle = \sum_j \beta_j |u_j\rangle$, gives

$$A^{-1}|b\rangle \;=\; \sum_j \frac{\beta_j}{\lambda_j}\,|u_j\rangle. \tag{1}$$

Solving the system is nothing more than dividing every eigen-component of b by its eigenvalue. HHL performs this division without being told the eigenvalues or eigenvectors, and outputs the normalised state $|x\rangle = A^{-1}|b\rangle / \|A^{-1}|b\rangle\|$. If A is not Hermitian one solves the enlarged Hermitian system with matrix $\left(\begin{smallmatrix} 0 & A \\ A^\dagger & 0 \end{smallmatrix}\right)$ instead [12]; if N is not a power of two the system is padded (Section 3.3).

2.3 Quantum phase estimation

Phase estimation (QPE) is the tool that reads eigenvalues. Given a unitary U and one of its eigenvectors, $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, QPE writes an n-bit approximation of φ into an n-qubit *clock* register: Hadamard gates put the clock into a uniform superposition, clock qubit j then controls U^{2^j}, and an inverse quantum Fourier transform (QFT) converts the accumulated phases into a binary number [10, 18]. For HHL we take $U = e^{iAt}$, whose eigenvectors are those of A and whose phases are $\varphi_j = \lambda_j t / 2\pi$. The clock then holds the integer

$$k_j \;=\; 2^n \varphi_j \;=\; \frac{\lambda_j\, t\, 2^n}{2\pi}. \tag{2}$$

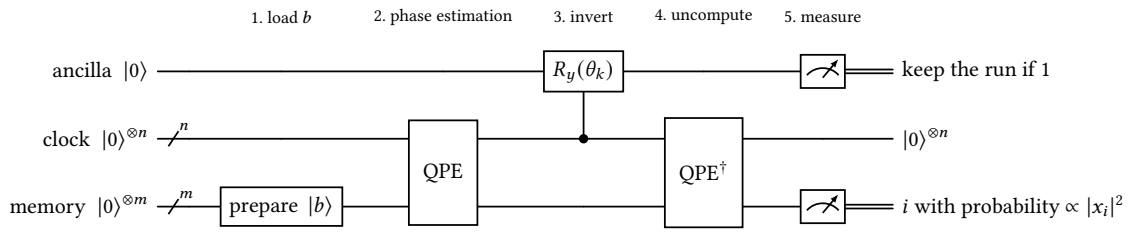


Fig. 1. The HHL circuit. The memory register holds $|b\rangle$ at the start and $|x\rangle$ at the end, the clock register temporarily stores the eigenvalues, and one ancilla qubit carries the factor $1/\lambda$.

If k_j is an integer the clock ends exactly in $|k_j\rangle$. If not, the outcome is spread over the integers around k_j; the nearest one still has probability at least $4/\pi^2 \approx 0.41$ [10]. This spreading is the only approximation in the circuits of this paper, and Section 5.6 measures its effect. Because QPE is linear, running it on $|b\rangle$ treats all eigen-components at once and produces $\sum_j \beta_j |k_j\rangle|u_j\rangle$.

2.4 The five stages of HHL

Figure 1 shows the circuit. It uses three registers: an ancilla qubit, the n-qubit clock and the m-qubit memory. Writing the registers in that order, the state evolves as follows.

(1) *Load b.* The memory is prepared in $|b\rangle$: $|0\rangle|0\rangle|b\rangle$.
(2) *Phase estimation.* The eigenvalues are written into the clock: $|0\rangle \sum_j \beta_j |k_j\rangle|u_j\rangle$.
(3) *Eigenvalue inversion.* The ancilla is rotated about the y axis by an angle that depends on the clock value, $\theta_k = 2\arcsin(C/\lambda_k)$, where λ_k is the eigenvalue that clock value k stands for. Since $R_y(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle$, this gives

$$\sum_j \beta_j \left(\sqrt{1 - C^2/\lambda_j^2}\,|0\rangle + \frac{C}{\lambda_j}|1\rangle \right) |k_j\rangle|u_j\rangle. \tag{3}$$

(4) *Uncompute.* Phase estimation is run backwards, which returns the clock to $|0\rangle$ and removes the labels $|k_j\rangle$.
(5) *Measure.* The ancilla is measured. With probability

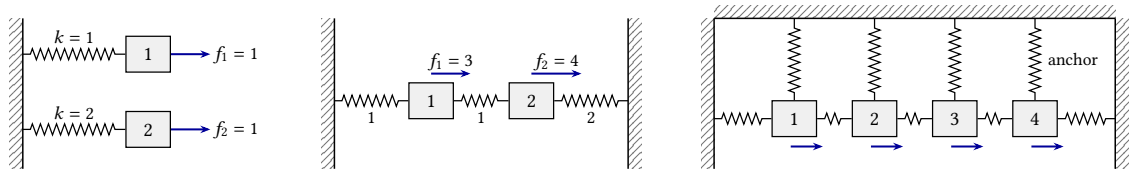
$$p_1 \;=\; C^2 \sum_j \frac{|\beta_j|^2}{\lambda_j^2} \;=\; C^2 \,\|A^{-1}|b\rangle\|^2 \tag{4}$$

the outcome is 1, and the memory is then left in $\sum_j (\beta_j/\lambda_j)|u_j\rangle$ up to normalisation, which is $|x\rangle$ by Eq. (1). Runs with outcome 0 are discarded.

Stage 4 is easy to overlook and essential. Without it every term in the memory would still be tagged by its own clock state $|k_j\rangle$, the terms could not interfere, and measuring the memory would give the probabilities of a mixture of eigenvectors instead of those of the single vector $\sum_j (\beta_j/\lambda_j)|u_j\rangle$.

2.5 Cost and fine print

For an s-sparse matrix, HHL prepares $|x\rangle$ to precision ϵ in time $\tilde{O}(\log(N)\, s^2\kappa^2/\epsilon)$ [12]; later algorithms improve the dependence on ϵ to polylogarithmic [7]. Even for the largest allowed C the success probability p_1 can be as small as $1/\kappa^2$; amplitude amplification reduces the number of repetitions this costs from about κ^2 to about κ [6, 12]. None of



(a) E1: two uncoupled springs (b) E2: two coupled blocks (c) E3: chain of four blocks, all $k = 1$, all $f_i = 1$

Fig. 2. The mechanical systems behind the experiments. Blocks slide horizontally; k is a spring stiffness and f_i the force applied to block i. In (c) every block is also tied to a fixed frame by an anchor spring, drawn vertically to keep the picture readable; it pulls block i back with force x_i. Experiment E4 uses the same chain with three blocks and a force on the first block only.

this helps if loading b or reading out x costs time proportional to N [1]. In a simulation of a 4×4 system none of these costs bite, but each of them leaves a visible trace in our code, and we point them out as they appear.

3 ENCODING A LINEAR SYSTEM INTO A CIRCUIT

3.1 The test problems

We wanted examples whose answers can be checked by intuition as well as by NumPy, so all four systems describe blocks connected by springs (Figure 2). If block i is displaced by x_i and pushed by a force f_i, balance of forces gives the linear system $Kx = f$. The *stiffness matrix* K carries on its diagonal the total stiffness of all springs attached to a block and off the diagonal minus the stiffness of the spring joining two blocks:

$$K_{\mathrm{E1}} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \qquad K_{\mathrm{E2}} = \begin{pmatrix} 2 & -1 \\ -1 & 3 \end{pmatrix}, \qquad K_{\mathrm{E3}} = \begin{pmatrix} 3 & -1 & 0 & 0 \\ -1 & 3 & -1 & 0 \\ 0 & -1 & 3 & -1 \\ 0 & 0 & -1 & 3 \end{pmatrix}. \tag{5}$$

In HHL's notation $A = K$ and $b = f$.

- **E1, two uncoupled springs.** $f = (1, 1)$, so $x = (1, \frac{1}{2})$: the stiffer spring stretches half as far. The matrix is diagonal, its eigenvectors are the basis states $|0\rangle$ and $|1\rangle$ and its eigenvalues can be read off, so every step of the algorithm can be checked by hand.
- **E2, two coupled blocks.** $f = (3, 4)$ and $x = (2.6,\ 2.2)$. The off-diagonal entry couples the blocks, the eigenvalues $(5 \mp \sqrt{5})/2$ are irrational, and our code never uses them.
- **E3, chain of four blocks.** $f = (1, 1, 1, 1)$ and $x = (0.6,\ 0.8,\ 0.8,\ 0.6)$: the blocks next to the walls move less than the two in the middle.
- **E4, chain of three blocks.** The 3×3 version of K_{E3} with $f = (1, 0, 0)$ and $x = (8, 3, 1)/21 \approx (0.381,\ 0.143,\ 0.048)$: a push on the first block dies out along the chain. Three is not a power of two, so this system is padded to 4×4.

Stiffness matrices of this kind have two convenient properties. They are symmetric and positive definite, so they are Hermitian with positive eigenvalues. And because their off-diagonal entries are never positive, their inverses have no negative entries [4]: pushing blocks to the right never moves a block to the left. We will use the second fact when reading out the solution, because a measurement only reveals $|x_i|$.

3.2 Encoding b: amplitude encoding

The right-hand side becomes the amplitudes of the memory register,

$$|b\rangle = \frac{1}{\|b\|} \sum_{i=1}^{N} b_i |i\rangle, \tag{6}$$

so the first entry sits on the bit string of all zeros, the second on $0 \ldots 01$, and so on. A quantum state has unit length, so the length $\|b\|$ is not stored in the circuit; we keep it on paper and multiply it back in at the end (Section 4.3). For one qubit a single rotation does the job, since $R_y(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle$:

$$\theta = 2\,\mathrm{atan2}(b_2, b_1). \tag{7}$$

For E1 this is $\theta = \pi/2$, the same state a Hadamard gate would give. For E2 the forces $(3, 4)$ have length 5, the amplitudes are $(0.6,\ 0.8)$ and $\theta = 2\arctan(4/3) = 1.8546$ rad.

For more qubits the same idea is applied recursively [17, 20]. The first qubit is rotated so that the probability of finding it in 0 or 1 equals the weight of the upper and the lower half of b. The second qubit then splits each half again, with one rotation angle for each value of the first qubit, and so on down a binary tree. Signs are taken care of in the last level by using the signed entries in the atan2. Our Cirq code builds this tree by hand (`amplitude_encoding`); in Qiskit the library gate `StatePreparation` does the same. For E3 the tree reduces to $R_y(\pi/2)$ on both qubits, and for E4 to no gate at all, because $(1, 0, 0, 0)$ is the state $|00\rangle$ the register starts in. A general vector needs on the order of N rotations, which is the first piece of fine print: HHL only pays off if b has enough structure to be loaded faster.

3.3 Encoding A: a time evolution

The matrix does not appear in the circuit as a table of numbers. It enters through the unitary $U = e^{iAt}$, and only in the controlled powers U^{2^j} that phase estimation needs. On a quantum computer these would be produced by a Hamiltonian-simulation routine that exploits the sparsity of A [5, 16]. On a simulator we take a shortcut: the $2^m \times 2^m$ matrix U is computed classically (the textbook listing diagonalises A, our Qiskit code calls a matrix exponential) and inserted as a single gate. This shortcut costs as much as solving the system classically, so it says nothing about speed. What it preserves is the logic of the algorithm, provided that the eigenvalues are not used anywhere else, and the next subsection makes sure of that.

When N is not a power of two, A is embedded in the next larger matrix with a constant λ_{pad} on the new diagonal entries and zeros elsewhere, and b is extended by zeros. The solution of the padded system is $(x, 0, \ldots, 0)$. For E4 we use $\lambda_{\mathrm{pad}} = 3$, the value of the other diagonal entries; any value inside the eigenvalue bounds of the next subsection works.

3.4 Choosing t and C without knowing the eigenvalues

HHL has two free parameters. The evolution time t sets the scale of the clock: by Eq. (2), clock value k stands for the eigenvalue

$$\lambda_k = \frac{2\pi k}{2^n t}, \qquad k = 1, \ldots, 2^n - 1. \tag{8}$$

Phases are periodic, so an eigenvalue above $2\pi/t$ would wrap around and be mistaken for a small one. The constant C scales the rotation of Eq. (3); the amplitude C/λ cannot exceed 1, so C must not be larger than the smallest eigenvalue. Both requirements ask for bounds on the spectrum, not for the spectrum itself:

$$C \le \lambda_{\min}, \qquad \lambda_{\max} \le \Lambda, \qquad t = \frac{2\pi}{\Lambda}\,\frac{2^n - 1}{2^n}. \tag{9}$$

Listing 1. Choosing the parameters and the rotation angles from eigenvalue bounds (`hhl_common.py`). Both implementations read their angles from this one table.

```python
def gershgorin_bounds(A):
    A = np.asarray(A)
    centre = np.real(np.diag(A))
    radius = np.sum(np.abs(A), axis=1) - np.abs(np.diag(A))
    return float(np.min(centre - radius)), float(np.max(centre + radius))

def evolution_time(lam_hi, n_clock):
    N = 2 ** n_clock
    return 2 * np.pi * (N - 1) / (N * lam_hi)

def clock_eigenvalues(n_clock, t):
    N = 2 ** n_clock
    k = np.arange(N, dtype=float)
    k[0] = N
    return 2 * np.pi * k / (N * t)

def rotation_angles(n_clock, t, C):
    ratio = C / clock_eigenvalues(n_clock, t)
    theta = np.zeros_like(ratio)
    ok = ratio <= 1.0 + 1e-9
    theta[ok] = 2 * np.arcsin(np.minimum(ratio[ok], 1.0))
    return theta
```

With this t the upper bound Λ sits on the last clock value $2^n - 1$, and neighbouring clock values are $\Lambda/(2^n - 1)$ apart.

Bounds of this kind are cheap. By the Gershgorin circle theorem every eigenvalue of A lies in at least one of the intervals $[a_{ii} - R_i,\ a_{ii} + R_i]$ with $R_i = \sum_{j \neq i} |a_{ij}|$ [13], which takes one pass over the entries of the matrix. For K_{E2} the two rows give $[1, 3]$ and $[2, 4]$, hence $C = 1$ and $\Lambda = 4$; the true eigenvalues, 1.38 and 3.62, lie inside as they must. For the two chains the rows give $[2, 4]$ and $[1, 5]$, hence $C = 1$ and $\Lambda = 5$. Listing 1 shows the corresponding code.

Three details of Listing 1 deserve a comment.

Clock value zero. Following the textbook listing, $k = 0$ is read as $k = 2^n$, that is, as the eigenvalue $2\pi/t$ just above Λ. Since phases are periodic, an eigenvalue close to Λ leaks to $k = 0$ as easily as to $k = 2^n - 2$, and this convention gives the leaked amplitude a sensible weight.

The textbook's default for C. The listing sets C to the smallest eigenvalue the clock can represent, $C = \lambda_1 = 2\pi/(2^n t)$. This is always allowed, but it ties C to the resolution of the clock: each extra clock qubit halves C and, by Eq. (4), divides the success probability by four. Taking C from a lower bound on the eigenvalues keeps the success probability independent of n.

Clock values below C. Once C is larger than λ_1, some clock values stand for eigenvalues smaller than C, for which $\arcsin(C/\lambda_k)$ does not exist. If C really is a lower bound, no eigenvalue of A lives there, and any amplitude found on those clock values has leaked in from phase estimation. We leave the ancilla untouched in that case (angle 0), in the spirit of the filter functions of the original paper [12].

The Gershgorin lower bound is positive only if every diagonal entry exceeds the sum of the absolute values of the other entries in its row. This holds for our spring systems because every block is tied to something fixed. For other matrices one has to bound $\lambda_{\min}$ by other means (the original algorithm assumes that κ is known [12]) or fall back on the textbook default. Table 1 lists the parameters of the four experiments. For E1, where the eigenvalues 1 and 2 are visible in the matrix, we chose $\Lambda = 3$ by hand so that with two clock qubits clock value k stands exactly for the eigenvalue k.

Table 1. The four experiments. C and Λ are the lower and upper eigenvalue bounds, n the number of clock qubits and t the evolution time of Eq. (9). The qubit count includes the ancilla.

	System	Size	b	$[C, \Lambda]$	Bounds from	n	t	Qubits
E1	Two uncoupled springs	2×2	$(1, 1)$	$[1, 3]$	by inspection	2	1.5708	4
E2	Two coupled blocks	2×2	$(3, 4)$	$[1, 4]$	Gershgorin	6	1.5463	8
E3	Chain of four blocks	4×4	$(1, 1, 1, 1)$	$[1, 5]$	Gershgorin	6	1.2370	9
E4	Chain of three blocks	3×3 (padded)	$(1, 0, 0)$	$[1, 5]$	Gershgorin	6	1.2370	9

Table 2. Changes made to the textbook listing. "Listing" refers to the files as printed, run with Cirq 1.7.0.

	In the listing	Effect	Our change
1	`HamiltonianSimulation` inherits from `cirq.SingleQubitGate`	`AttributeError` on import; the class was removed from Cirq	define `_num_qubits_`, which also lifts the restriction to one qubit
2	imports a `QFT` class from Chapter 4 of the book	needed to build phase estimation	rebuilt gate by gate from the circuit diagram in the book's output
3	`HHL(...)` is created without `t`	runs with $t = 1$; eigenvalues fall between clock values and the result is biased	pass `t=t`
4	rotation angle computed as `asin(C*N*t/(2*pi*k))`	`ValueError` (math domain error) as soon as $C > \lambda_1$	shared angle table with the cut-off of Section 3.4
5	memory register is a single qubit	only 2×2 systems	m memory qubits, amplitude encoding, counts over all basis states
6	unused `main()` that calls undefined names; a method `circuit()` that overwrites itself; debugging prints	harmless but confusing	removed or simplified; readable labels in circuit diagrams

4 IMPLEMENTATION

4.1 Cirq: from the textbook listing to a working program

Listing 5.1 of Pattanayak's book [19] implements HHL for a 2×2 matrix in Cirq [8]. It is closely related to the HHL example distributed with Cirq [9], with which it shares its test matrix and its rotation code, and which in turn cites the survey of Abhijith et al. [2]. The listing splits the program into classes that mirror Figure 1. The class `HamiltonianSimulation` provides $U = e^{-iAt}$ as a Cirq `EigenGate`. The classes `ControlledUnitary` and `QuantumPhaseEstimation` build stage 2, and `EigenValueInversion` is stage 3. The class `HHL` assembles the circuit and measures the Pauli observables $\langle X \rangle$, $\langle Y \rangle$ and $\langle Z \rangle$ of the memory qubit.

The listing was written for Cirq 0.8.2 and does not run on the current release (1.7.0). Table 2 lists what we changed. The first two rows are needed before the program starts at all. The third is the interesting one: the script defines the evolution time $t = 0.358166\,\pi$, then creates the solver without it,

```
_hhl_ = HHL(hamiltonian=A, initial_state_transforms=initial_state_transforms,
            qpe_register_size=4)                # as printed: t falls back to 1
_hhl_ = HHL(hamiltonian=A, initial_state_transforms=initial_state_transforms,
            qpe_register_size=4, t=t)           # corrected
```

so the constructor's default $t = 1$ is used. Nothing crashes; the circuit is simply built for the wrong time. Section 5.2 shows that this accounts for the numbers printed in the book.

Two conventions of the listing are worth knowing because they differ from most textbooks. First, its clock register is big-endian: the first clock qubit is the most significant bit and controls the highest power $U^{2^{n-1}}$. Second, it evolves

Listing 2. Eigenvalue inversion in Cirq, adapted from the textbook listing (`hhl_cirq.py`). The argument `qubits` holds the clock qubits followed by the ancilla.

```
def _decompose_(self, qubits):
    base_state = self.N - 1
    for eig_val_state in range(self.N):
        if eig_val_state != 0:
            base_state = eig_val_state - 1

        qubits_to_flip = eig_val_state ^ base_state

        for q in qubits[-2::-1]:
            if qubits_to_flip % 2 == 1:
                yield cirq.X(q)
            qubits_to_flip >>= 1

        theta = self.angles[eig_val_state]
        if theta == 0.0:
            continue

        eig_val_gate = cirq.ControlledGate(cirq.ry(theta),
                                           num_controls=self._num_qubits - 1)

        yield eig_val_gate(*qubits)
```

with $U = e^{-iAt}$ and compensates with a Fourier transform of the opposite sign: what the book calls the QFT multiplies by $e^{-2\pi i x y/2^n}$, and phase estimation applies its inverse. We confirmed this by comparing the rebuilt circuit with the discrete Fourier transform matrix. The combination stores the same integer k_j as Eq. (2).

Listing 2 shows how the listing implements stage 3. A controlled gate acts when all its control qubits are 1. To make it act on clock value k, the clock qubits that are 0 in the binary form of k are flipped with X gates first. Going through $k = 0, 1, 2, \ldots$ in order, only the bits in which k and $k-1$ differ need to be flipped between two rotations, and after the last value all flips have cancelled. The result is one multi-controlled R_y for every clock value that is allowed to rotate the ancilla.

4.2 Qiskit: written from scratch

Qiskit [15] used to include an `HHL` class, but it was deprecated and removed together with the other linear solvers of `qiskit.algorithms` [14], so the circuit has to be assembled by hand. Listing 3 is the core of our implementation; it was written with the help of an AI assistant (see the acknowledgments).

The code reads like Figure 1. Phase estimation is built as a sub-circuit so that stage 4 is one call to `inverse()`. Stage 3 is a single library gate: a *uniformly controlled rotation* `UCRYGate` takes a list of 2^n angles and applies the k-th one when its control qubits hold the value k. It is a look-up table in gate form and compiles to 2^n CNOT gates and 2^n single-qubit rotations [17], which is cheaper than the 2^n multi-controlled rotations of Listing 2.

Qiskit's conventions are the opposite of the listing's in almost every respect (Table 3). Registers are little-endian, so `clock[j]` controls U^{2^j}, and we use the textbook form of phase estimation, $U = e^{+iAt}$ followed by the inverse QFT. The two programs nevertheless prepare the same state. We did not take that on faith: for all four experiments and for every clock size in Section 5.6 the exact output probabilities of the two programs differ by at most 1.2×10^{-14}.

4.3 Reading out the solution

At the end of the circuit we measure the ancilla and the memory and count how often each combination occurs. Let $n(a, i)$ be the number of shots, out of S, in which the ancilla showed a and the memory showed its i-th basis state. Only

Listing 3. The HHL circuit in Qiskit (`hhl_qiskit.py`).

```python
def _phase_estimation(self):
    qpe = QuantumCircuit(self.memory, self.clock, name="QPE")
    qpe.h(self.clock)
    for j in range(self.n_clock):
        U = UnitaryGate(expm(1j * self.A * self.t * 2 ** j), label=f"U^{2 ** j}")
        qpe.append(U.control(1), [self.clock[j], *self.memory])
    qpe.append(QFTGate(self.n_clock).inverse(), self.clock)
    return qpe

def build_circuit(self):
    qpe = self._phase_estimation()
    qc = QuantumCircuit(self.memory, self.clock, self.ancilla)
    qc.append(StatePreparation(self.b, label="|b>"), self.memory)
    qc.append(qpe.to_gate(label="QPE"), [*self.memory, *self.clock])
    angles = rotation_angles(self.n_clock, self.t, self.C)
    rotation = UCRYGate(list(angles))
    rotation.label = "Ry(theta_k)"
    qc.append(rotation, [self.ancilla[0], *self.clock])
    qc.append(qpe.inverse().to_gate(label="QPE inverse"), [*self.memory, *self.clock])
    return qc
```

Table 3. Conventions of the two implementations.

	Cirq (from the listing)	Qiskit (ours)
Bit order in a register	first qubit is the most significant bit	first qubit is the least significant bit
Evolution operator	$U = e^{-iAt}$	$U = e^{+iAt}$
Clock qubit j controls	$U^{2^{n-1-j}}$	U^{2^j}
Transform after the controlled powers	phases $e^{+2\pi i xy/2^n}$ (the listing's "inverse QFT")	phases $e^{-2\pi i xy/2^n}$ (inverse of `QFTGate`)
Construction of U	eigendecomposition inside a `cirq.EigenGate`	`scipy.linalg.expm`, wrapped in a `UnitaryGate`
Loading b	tree of controlled R_y gates, written by hand	`StatePreparation`
Eigenvalue inversion	multi-controlled R_y per clock value, selected with X gates	one `UCRYGate` (look-up table)
Exact statistics	`cirq.Simulator().simulate`	`qiskit.quantum_info.Statevector`
Sampling	`cirq.Simulator().run`	`AerSimulator().run`
Printed outcomes	one array per measurement key	bit string, last classical bit first

$a = 1$ matters. Combining Eq. (4) with the fact that $A^{-1}|b\rangle = x/\|b\|$, the probability of the pair $(1, i)$ is

$$P(1,i) = \frac{C^2}{\|b\|^2}\, x_i^2 \qquad \Longrightarrow \qquad |x_i| \;\approx\; \frac{\|b\|}{C}\sqrt{\frac{n(1,i)}{S}}. \tag{10}$$

This one formula is the whole interpretation step, and three remarks complete it.

The norm is not lost. A normalised state cannot hold $\|x\|$, but the success rate can: summing Eq. (10) over i gives $\|x\| \approx (\|b\|/C)\sqrt{n_1/S}$, where n_1 is the number of accepted shots. The discarded runs are not wasted; their fraction is a measurement.

Error bars. Each count is binomial, so the standard error of $\hat{p} = n(1,i)/S$ is $\sqrt{\hat{p}(1-\hat{p})/S}$, and propagating it through the square root gives

$$\delta|x_i| = \frac{\|b\|}{C}\sqrt{\frac{1-\hat{p}}{4S}} \;\le\; \frac{\|b\|}{2C\sqrt{S}}. \tag{11}$$

The absolute uncertainty is roughly the same for every entry, so small entries are the hardest to measure in relative terms. All ± values in this paper are one standard error computed in this way.

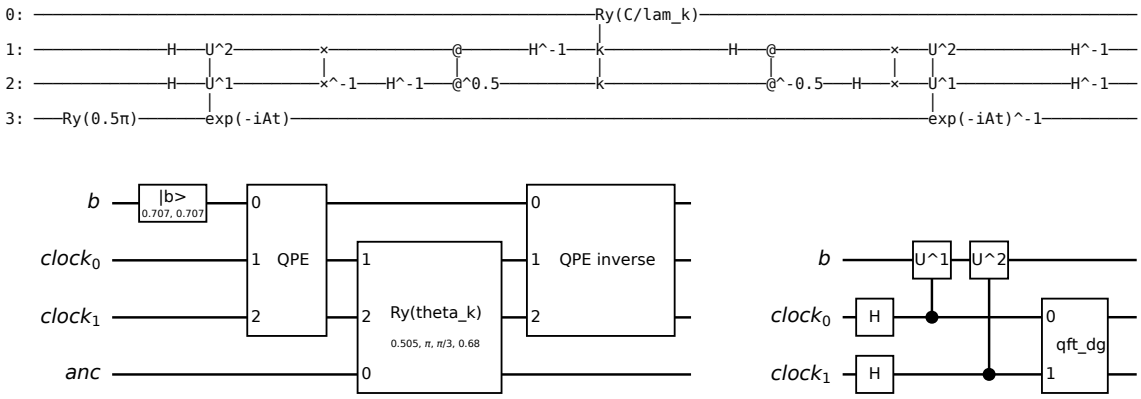


Fig. 3. The circuit of experiment E1 as drawn by each framework. Top: Cirq, with the ancilla on line 0, the clock on lines 1 and 2 and the memory on line 3; `H^-1` and `x^-1` are the inverses of the Hadamard and swap gates, which equal the gates themselves. Bottom left: Qiskit, with the memory on top and the angles $\theta_0, \ldots, \theta_3$ of the look-up rotation printed in the box. Bottom right: the inside of the QPE block.

Signs. Eq. (10) only gives magnitudes. For our spring systems this is enough, because all displacements are known to be non-negative (Section 3). In general, relative signs and phases need additional measurements in other bases. For a one-qubit memory the textbook listing does this by measuring the three Pauli observables: if $|x\rangle = \tilde{x}_1|0\rangle + \tilde{x}_2|1\rangle$, then $\langle Z\rangle = |\tilde{x}_1|^2 - |\tilde{x}_2|^2$ fixes the magnitudes and $\langle X\rangle = 2\,\mathrm{Re}(\tilde{x}_1^*\tilde{x}_2)$ and $\langle Y\rangle = 2\,\mathrm{Im}(\tilde{x}_1^*\tilde{x}_2)$ fix the relative phase. We keep this read-out for the 2×2 experiments. A positive $\langle X\rangle$ together with $\langle Y\rangle = 0$ means that the two entries are real with the same sign.

On a simulator there is a second way to look at the result: the final state vector gives the probabilities $P(1, i)$ exactly, without shot noise. We report both, the "exact circuit" values to show what the algorithm itself gets wrong, and sampled values to show what an experiment would see. From the state vector we also compute the fidelity $F = \langle x|\rho|x\rangle$ between the classical solution and the state ρ of the memory after a successful run.

4.4 Testing

Both programs come with 60 automated tests. They compare each building block with plain NumPy (the evolution gate with a matrix exponential, the Fourier transform with its matrix, the clock contents and rotation angles with their formulas), each complete solver with `numpy.linalg.solve`, and the two frameworks with each other. As a check that does not depend on either framework, the five stages of Section 2.4 are also written out as plain matrix algebra in about twenty lines of NumPy; both circuits reproduce its output probabilities. All figures, tables and numbers quoted in this paper are generated from one results file by a script, so nothing was copied by hand.

5 EXPERIMENTS

5.1 Setup

All runs are noiseless simulations with Python 3.11.15, Cirq 1.7.0, Qiskit 2.5.2 and Qiskit Aer 0.17.2. Each circuit is sampled 8192 times. The random seed is fixed (2026) and the Qiskit circuits are compiled without optimisation; with these two settings and the library versions above, repeated runs reproduce every number in this paper exactly (Section 6

Table 4. The textbook example: Pauli expectation values of the solution qubit. "Exact" rows are computed from the state vector; sampled rows show one standard error.

Source	$\langle X\rangle$	$\langle Y\rangle$	$\langle Z\rangle$
Expected values quoted in the listing	0.1441	0.4132	−0.8992
Classical solution (NumPy)	0.1441	0.4132	−0.8992
Output printed in the book (5000 repetitions)	0.2128	0.3746	−0.8580
Original listing, unmodified (mean of 100 runs)	0.2025 ± 0.0030	0.3880 ± 0.0023	−0.8651 ± 0.0013
Our port with t not passed ($t = 1$), exact	0.2025	0.3876	−0.8642
Our port with t passed, exact	0.1441	0.4132	−0.8992
Cirq with t passed, 5000 repetitions	0.145 ± 0.027	0.411 ± 0.026	−0.894 ± 0.012
Qiskit with t passed, 5000 repetitions	0.131 ± 0.027	0.414 ± 0.025	−0.904 ± 0.012

explains why the second setting is needed). The complete set of experiments takes well under a minute on two CPU cores.

5.2 Validation on the textbook example

The listing comes with its own test case: a 2×2 Hermitian matrix with eigenvalues 0.349 and 4.537, a complex $|b\rangle$ prepared by two rotations, four clock qubits, and the expected values $\langle X\rangle = 0.1441$, $\langle Y\rangle = 0.4132$ and $\langle Z\rangle = -0.8992$ of the solution qubit. The time $t = 0.358166\,\pi$ is chosen so that the two eigenvalues land exactly on the clock values 1.00 and 13.00. The output printed in the book, however, is $(0.2128,\ 0.3746,\ -0.8580)$.

Table 4 resolves the discrepancy. To rule out our own port as the cause, we first ran the original files unmodified, supplying the two missing pieces of Table 2 (rows 1 and 2) from outside. Averaged over 100 runs they print $(0.203,\ 0.388,\ -0.865)$, which is 27 standard errors away from the values the listing expects and matches, within 0.7 standard errors, the exact result of our port when t is left at its default of 1. In the Cirq example the time is passed to the circuit; the argument was lost when the code was reorganised into classes. With $t = 1$ the eigenvalues correspond to the clock values 0.89 and 11.55, in between the integers, and phase estimation smears them over several clock values. The book's single run lies within 0.5 standard errors of this prediction and 3.4 standard errors from the intended values. Its digits even reveal the sample size: each printed value is $1 - 2n_1/n_{\text{acc}}$, and the three numbers are reproduced exactly by $n_{\text{acc}} = 1278$, 1292 and 1296 accepted runs out of 5000, in line with the predicted success probability of 0.259.

Once t is passed on, the port reproduces the expected values to four decimal places, the Qiskit implementation gives the same exact result, and both sampled runs agree within their error bars. This is the validation we rely on for the rest of the paper.

5.3 E1: a diagonal system

E1 is small enough to follow by hand, and Figure 3 shows its circuit as drawn by both frameworks. The input is visible in the drawings: b is the gate $R_y(0.5\pi)$ on the memory line of the Cirq diagram and the box with amplitudes $0.707,\ 0.707$ in the Qiskit diagram, and the matrix appears only through the controlled powers of U. The memory starts in $(|0\rangle + |1\rangle)/\sqrt{2}$. Its two basis states are the eigenvectors of K_{E1} with eigenvalues 1 and 2, and with $t = \pi/2$ and two clock qubits phase estimation stores exactly $k = 1$ and $k = 2$. The rotation angles are $\theta_1 = 2\arcsin 1 = \pi$ and $\theta_2 = 2\arcsin\frac{1}{2} = \pi/3$, the two middle entries printed in the rotation box of Figure 3. After uncomputation the ancilla and the memory are in

$$\tfrac{1}{\sqrt{2}}\,|1\rangle|0\rangle + \tfrac{1}{\sqrt{2}}\left(\tfrac{\sqrt{3}}{2}|0\rangle + \tfrac{1}{2}|1\rangle\right)|1\rangle, \tag{12}$$

Table 5. Results of the two 2×2 experiments. "Classical" is `numpy.linalg.solve`; "Circuit, exact" is computed from the state vector; the last two columns are decoded from measurement counts with Eq. (10). The success probability in the classical column is the ideal value of Eq. (4).

	Classical	Circuit, exact	Cirq, 8192 shots	Qiskit, 8192 shots
E1: two uncoupled springs				
x_1	1.000	1.000	0.994 ± 0.006	1.004 ± 0.006
x_2	0.500	0.500	0.504 ± 0.007	0.502 ± 0.007
$\|x\|$	1.118	1.118	1.115 ± 0.005	1.122 ± 0.005
success probability	0.625	0.625	0.621 ± 0.005	0.630 ± 0.005
$\langle X \rangle$	0.800	0.800	0.793 ± 0.009	0.795 ± 0.008
$\langle Y \rangle$	0.000	0.000	−0.016 ± 0.014	0.001 ± 0.014
$\langle Z \rangle$	0.600	0.600	0.610 ± 0.011	0.582 ± 0.011
E2: two coupled blocks				
x_1	2.600	2.570	2.523 ± 0.024	2.578 ± 0.024
x_2	2.200	2.179	2.194 ± 0.025	2.183 ± 0.025
$\|x\|$	3.406	3.369	3.343 ± 0.021	3.378 ± 0.020
success probability	0.464	0.454	0.447 ± 0.005	0.456 ± 0.006
$\langle X \rangle$	0.986	0.986	0.984 ± 0.003	0.988 ± 0.003
$\langle Y \rangle$	0.000	0.000	−0.015 ± 0.016	−0.008 ± 0.017
$\langle Z \rangle$	0.166	0.164	0.189 ± 0.016	0.164 ± 0.016

so the four outcomes (ancilla bit, memory bit) = $(1,0)$, $(1,1)$, $(0,1)$ and $(0,0)$ have probabilities $\frac{1}{2}$, $\frac{1}{8}$, $\frac{3}{8}$ and 0. Memory bit 0 belongs to x_1 and memory bit 1 to x_2. The Cirq run returned 4047, 1042, 3103 and 0 counts for these four outcomes, and Eq. (10) with $\|b\| = \sqrt{2}$ and $C = 1$ turns the first two into

$$|x_1| \approx \sqrt{2}\,\sqrt{4047/8192} = 0.994, \qquad |x_2| \approx \sqrt{2}\,\sqrt{1042/8192} = 0.504. \tag{13}$$

The two frameworks present these raw results differently. Cirq returns one array of bits per measurement, which we tally into the four counts above. Qiskit returns a dictionary of bit strings, for this experiment `{'10': 4126, '01': 3033, '11': 1033}`, in which the last classical bit is printed first: the left character is the ancilla and the right one the memory. For a larger memory the remaining characters, read as a binary number, give the position of the entry, counted from zero.

Table 5 compares both frameworks with the classical solution $(1, \frac{1}{2})$. Because the eigenvalues sit exactly on clock values (Figure 4a), the circuit itself is exact and the only error is shot noise.

5.4 E2: a coupled system with unknown eigenvalues

For E2 the code is given only the matrix and the right-hand side. It derives $C = 1$ and $\Lambda = 4$ from the Gershgorin intervals, sets $t = 1.5463$ by Eq. (9) and uses 6 clock qubits, so neighbouring clock values are 0.063 apart. Figure 4b shows what phase estimation finds. The larger eigenvalue happens to fall almost on a clock value ($k = 56.98$) and appears as a single bar. The smaller one falls at $k = 21.77$: the nearest clock value 22 receives probability 0.72 and the rest spreads over the neighbours. Reading the histogram against the upper axis gives the eigenvalues to two digits. This is the sense in which HHL works "without knowing the eigenvalues": the clock register measures them on the fly.

The consequence of the spreading is visible in Table 5. Even without shot noise the circuit returns $(2.570,\ 2.179)$ where the classical solution is $(2.6,\ 2.2)$, an error of 1.1 % of $\|x\|$. Both entries are about 1 % low, so the direction of the solution is nearly perfect (fidelity 0.9998) and the error sits almost entirely in the norm, which is recovered from a

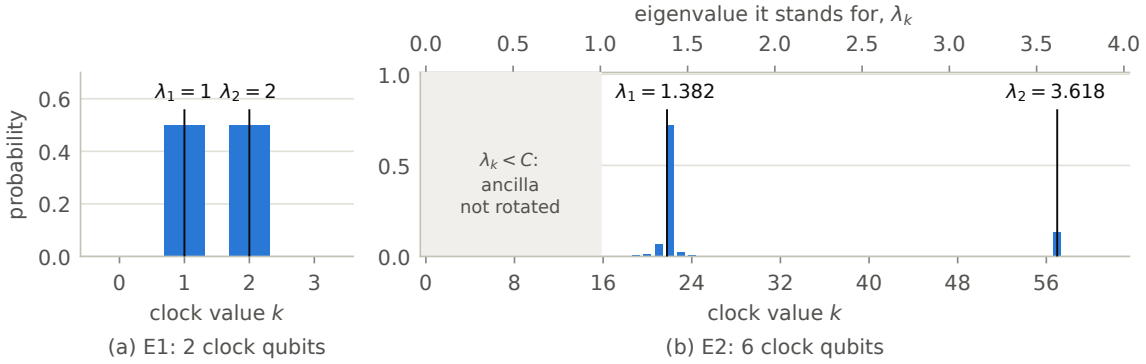


Fig. 4. Contents of the clock register after phase estimation. (a) In E1 the two eigenvalues fall exactly on clock values. (b) In E2 the eigenvalue λ_1 falls between two clock values and spreads over its neighbours. The vertical lines mark the true eigenvalues, which the algorithm is never told. In the shaded region the ancilla is not rotated.

Table 6. Results of the experiments with a two-qubit memory. Columns as in Table 5. The padding state of E4 is the fourth basis state of the memory, which does not correspond to any block.

	Classical	Circuit, exact	Cirq, 8192 shots	Qiskit, 8192 shots
E3: chain of four blocks				
x_1	0.600	0.597	0.576 ± 0.011	0.574 ± 0.011
x_2	0.800	0.796	0.800 ± 0.010	0.792 ± 0.010
x_3	0.800	0.796	0.791 ± 0.010	0.823 ± 0.010
x_4	0.600	0.597	0.604 ± 0.011	0.598 ± 0.011
$\|\|x\|\|$	1.414	1.407	1.401 ± 0.008	1.411 ± 0.008
success probability	0.500	0.495	0.491 ± 0.006	0.498 ± 0.006
E4: chain of three blocks				
x_1	0.381	0.381	0.385 ± 0.005	0.382 ± 0.005
x_2	0.143	0.143	0.139 ± 0.005	0.131 ± 0.005
x_3	0.048	0.050	0.059 ± 0.006	0.054 ± 0.006
padding state	0	0.000	0 counts	0 counts
$\|\|x\|\|$	0.410	0.410	0.414 ± 0.005	0.407 ± 0.005
success probability	0.168	0.168	0.171 ± 0.004	0.166 ± 0.004

success probability of 0.454 instead of the ideal 0.464. The sampled values of both frameworks scatter around the exact circuit values as the error bars predict, and the positive $\langle X \rangle$ with vanishing $\langle Y \rangle$ confirms that both displacements have the same sign.

5.5 E3 and E4: larger systems

Nothing in either program is specific to 2×2 matrices, so the step to a two-qubit memory only changes the input. For the chain of four blocks (E3) the uniform load is encoded by $R_y(\pi/2)$ on both memory qubits, and the Gershgorin bounds are $[1, 5]$. With six clock qubits the exact circuit output is within 0.5 % of the classical solution $(0.6,\ 0.8,\ 0.8,\ 0.6)$, and the sampled values reproduce the symmetric profile of the chain (Table 6, Figure 5).

E4 exercises padding. The 3×3 chain is embedded in a 4×4 matrix, and the fourth basis state of the memory is never observed in either framework, as it should be. E4 also shows the read-out problem in miniature. The third displacement

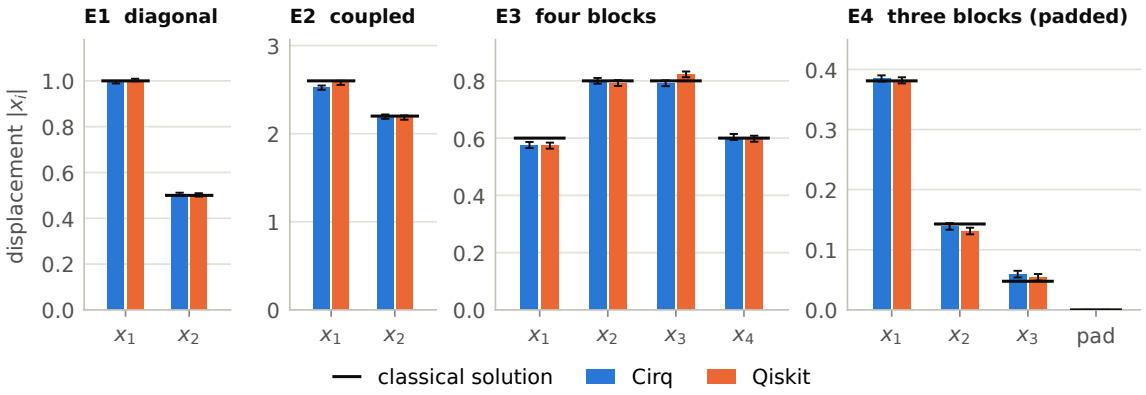


Fig. 5. Displacements decoded from 8192 shots in Cirq and Qiskit, compared with the classical solution (black lines). Error bars show one standard error.

Table 7. Summary of the four circuits. Success probability, fidelity and relative error are exact (state vector) values; gate counts refer to the Qiskit circuit compiled to CNOT and single-qubit gates.

	E1	E2	E3	E4
Qubits	4	8	9	9
Success probability	0.625	0.454	0.495	0.168
Fidelity with the classical solution	1.0000	0.9998	0.9997	0.9972
Relative error of $\lvert x\rvert$	0.00 %	1.07 %	0.51 %	0.68 %
CNOT gates after compilation	20	166	238	262
Circuit depth after compilation	37	243	370	418

is only 0.048, less than ten times the uncertainty $\|b\|/(2C\sqrt{S}) = 0.0055$ that Eq. (11) assigns to every entry, and with a success probability of 0.168 only about one shot in six is accepted. Its sampled values carry a relative uncertainty of about 10 %, while the largest entry is known to about 1 %. Over all four experiments the largest deviation between a sampled entry and the exact circuit value is 2.6 standard errors among 22 entries, as expected for statistical scatter. Table 7 summarises the algorithmic accuracy and the size of the four circuits.

5.6 Accuracy and cost versus clock size

How many clock qubits are enough? Figure 6 repeats E2 to E4 for $n = 2, \dots, 9$ and plots the error of the exact circuit output, so that shot noise plays no role. The curves are not smooth, because the error depends on where the eigenvalues happen to fall between clock values, and that changes with n. The trend is clear nevertheless: a least-squares fit gives a reduction by a factor of 0.61 (E2), 0.46 (E3) and 0.52 (E4) per additional qubit, that is, the error roughly halves. This is the $1/\epsilon$ in the running time of HHL seen from the other side: one more bit of accuracy needs one more clock qubit, and the longest evolution $U^{2^{n-1}}$ doubles.

The price is on the right of Figure 6. Between six and nine clock qubits the CNOT count of the compiled circuit grows by a factor of 1.63 (E2), 1.51 (E3) and 1.49 (E4) per qubit, heading towards 2, because the look-up rotation has one angle per clock value. For E2 the error falls from 2.12 % at four clock qubits (68 CNOTs) to 0.09 % at eight (424 CNOTs).

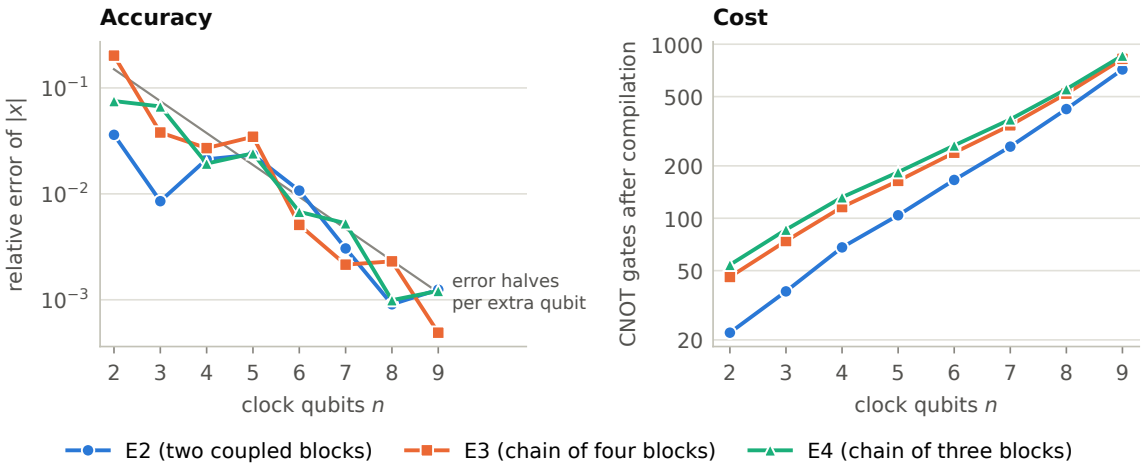


Fig. 6. Left: relative error of the exact circuit output, $\| \, |x_{\mathrm{HHL}}| - |x| \, \| / \|x\|$, against the number of clock qubits. Right: CNOT gates of the compiled Qiskit circuit.

A rotation that computes $\arcsin(C/\lambda)$ with quantum arithmetic instead of a table would avoid the exponential growth, at the cost of a far more complex circuit.

6 DISCUSSION

6.1 Lessons for the next student

Fix the conventions before writing gates. Bit order and the sign of the evolution have to match the Fourier transform. A mismatch does not raise an error; with the wrong sign the clock simply holds $2^n - k$ instead of k and every eigenvalue is inverted as if it were a different one. Table 3 is the table we wish we had started with.

Distrust default arguments. The mistake in the textbook example is one missing keyword. The program ran, produced plausible numbers and was printed. Comparing with a classical solution is what exposes it.

Look at the clock register first. A histogram like Figure 4 can be produced after stage 2 alone. If its peaks are not where the eigenvalue bounds say they should be, nothing after it can work, and the histogram says whether t, the bit order or the sign is at fault.

Separate the two kinds of error. The state vector shows the error of the algorithm; sampling adds shot noise on top. Looking only at sampled results, the 1 % bias of E2 and a statistical fluctuation are indistinguishable.

Choose C on purpose. The default of the listing is safe but makes the success probability collapse as the clock grows. Any lower bound on the eigenvalues is better.

A fixed seed is not always enough. Rebuilding this paper from scratch changed one sampled number although every seed was fixed. The cause was the compilation step in front of the Qiskit simulator: with its default settings the transpiler re-synthesises groups of gates and chooses among equivalent decompositions at random, and the simulator then draws a different, equally valid sample. Compiling with `optimization_level=0` removed the variation. We only found this because we rebuilt everything twice and compared the results file byte by byte.

6.2 Limitations

Our circuits contain e^{iAt} as a ready-made matrix, computed classically, and therefore demonstrate the logic of HHL, not its speed. The matrices are positive definite; negative eigenvalues would require reading the upper half of the clock values as negative numbers. The parameter recipe depends on a positive Gershgorin bound, which not every matrix offers. The read-out of Section 4.3 recovers magnitudes only and needs a number of shots that grows with the number of entries one wants to know. Finally, the simulations are noiseless. The compiled circuits contain between 20 and 262 CNOT gates (Table 7), and we have not tested how they would fare on hardware.

7 CONCLUSION

We built the HHL algorithm in Cirq and in Qiskit and used it to solve four small linear systems. Starting from a textbook listing forced us to understand every line of it: repairing it for a current Cirq release uncovered an unpassed argument that explains why the numbers printed in the book do not match the numbers the listing itself expects. The two implementations use opposite conventions in almost every respect and still agree to machine precision, which is the strongest test we have. With eigenvalue bounds from the Gershgorin theorem in place of the eigenvalues, six clock qubits are enough to recover the solutions of all four systems to about 1 % before shot noise, and every further clock qubit halves the error while the look-up rotation makes the circuit about one and a half times as large. Natural next steps are to replace the classically computed evolution by a genuine Hamiltonian simulation, to add amplitude amplification, and to run the four-qubit circuit of E1 on real hardware.

ACKNOWLEDGMENTS

Use of AI. The assignment allows AI help for the Qiskit implementation, with attribution. Claude, a large language model by Anthropic [3], was used to write the Qiskit implementation. It was also used to port the textbook listing to the current Cirq release, to write the experiment, plotting and test scripts, and to prepare a first draft of this text. The Cirq code starts from Listing 5.1 of Pattanayak [19]. The author is responsible for the final content.

REFERENCES

[1] Scott Aaronson. 2015. Read the fine print. *Nature Physics* 11, 4 (2015), 291–293. https://doi.org/10.1038/nphys3272

[2] Abhijith J., Adetokunbo Adedoyin, John Ambrosiano, et al. 2022. Quantum Algorithm Implementations for Beginners. *ACM Transactions on Quantum Computing* 3, 4, Article 18 (2022), 92 pages. https://doi.org/10.1145/3517340

[3] Anthropic. 2026. Claude (model identifier `claude-fable-5-1`). Large language model. https://claude.ai Used on September 30, 2026.

[4] Abraham Berman and Robert J. Plemmons. 1994. *Nonnegative Matrices in the Mathematical Sciences.* Classics in Applied Mathematics, Vol. 9. Society for Industrial and Applied Mathematics, Philadelphia, PA. https://doi.org/10.1137/1.9781611971262

[5] Dominic W. Berry, Graeme Ahokas, Richard Cleve, and Barry C. Sanders. 2007. Efficient Quantum Algorithms for Simulating Sparse Hamiltonians. *Communications in Mathematical Physics* 270, 2 (2007), 359–371. https://doi.org/10.1007/s00220-006-0150-x

[6] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. 2002. Quantum amplitude amplification and estimation. In *Quantum Computation and Information.* Contemporary Mathematics, Vol. 305. American Mathematical Society, Providence, RI, 53–74. https://doi.org/10.1090/conm/305/05215

[7] Andrew M. Childs, Robin Kothari, and Rolando D. Somma. 2017. Quantum Algorithm for Systems of Linear Equations with Exponentially Improved Dependence on Precision. *SIAM J. Comput.* 46, 6 (2017), 1920–1950. https://doi.org/10.1137/16M1087072

[8] Cirq Developers. 2026. Cirq. https://doi.org/10.5281/zenodo.4062499 Version 1.7.0 used in this work.

[9] Cirq Developers. 2026. HHL algorithm example (`examples/hhl.py`). Cirq source repository. https://github.com/quantumlib/Cirq/blob/main/examples/hhl.py Accessed September 30, 2026.

[10] Richard Cleve, Artur Ekert, Chiara Macchiavello, and Michele Mosca. 1998. Quantum algorithms revisited. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 454, 1969 (1998), 339–354. https://doi.org/10.1098/rspa.1998.0164

[11] Danial Dervovic, Mark Herbster, Peter Mountney, Simone Severini, Naïri Usher, and Leonard Wossnig. 2018. Quantum linear systems algorithms: a primer. arXiv:1802.08227 [quant-ph] https://arxiv.org/abs/1802.08227

[12] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. 2009. Quantum Algorithm for Linear Systems of Equations. *Physical Review Letters* 103, 15 (2009), 150502. https://doi.org/10.1103/PhysRevLett.103.150502

[13] Roger A. Horn and Charles R. Johnson. 2012. *Matrix Analysis* (2nd ed.). Cambridge University Press, Cambridge, UK.

[14] IBM Quantum. [n. d.]. Algorithms migration guide. IBM Quantum Documentation. https://quantum.cloud.ibm.com/docs/en/migration-guides/qiskit-algorithms-module Accessed September 30, 2026.

[15] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. https://doi.org/10.48550/arXiv.2405.08810 arXiv:2405.08810 [quant-ph]

[16] Seth Lloyd. 1996. Universal Quantum Simulators. *Science* 273, 5278 (1996), 1073–1078. https://doi.org/10.1126/science.273.5278.1073

[17] Mikko Möttönen, Juha J. Vartiainen, Ville Bergholm, and Martti M. Salomaa. 2005. Transformation of quantum states using uniformly controlled rotations. *Quantum Information and Computation* 5, 6 (2005), 467–473. https://doi.org/10.26421/QIC5.6-5

[18] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum Computation and Quantum Information* (10th anniversary ed.). Cambridge University Press, Cambridge, UK. https://doi.org/10.1017/CBO9780511976667

[19] Santanu Pattanayak. 2021. *Quantum Machine Learning with Python: Using Cirq from Google Research and IBM Qiskit.* Apress, Berkeley, CA. https://doi.org/10.1007/978-1-4842-6522-2

[20] Vivek V. Shende, Stephen S. Bullock, and Igor L. Markov. 2006. Synthesis of quantum-logic circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25, 6 (2006), 1000–1010. https://doi.org/10.1109/TCAD.2005.855930

A REPRODUCING THE RESULTS

The supplementary archive contains the code, the unmodified textbook files and this paper's LaTeX source. With Python 3.11 or newer:

```
pip install cirq-core qiskit qiskit-aer matplotlib pylatexenc pytest
cd code
pytest -q                     # automated checks
python run_experiments.py     # all experiments -> ../results/results.json
python make_paper_assets.py   # figures, tables and numbers for the paper
python textbook_replay.py     # runs the original textbook files unmodified
```

B SOURCE CODE

The three modules below are complete. The scripts that run the experiments and produce the figures are part of the supplementary archive.

Listing 4. `hhl_common.py`: test systems, parameter choice, rotation angles and decoding.

```
from dataclasses import dataclass

import numpy as np


@dataclass(frozen=True)
class LinearSystem:
    key: str
    title: str
    A: np.ndarray
    b: np.ndarray
    n_unknowns: int
    n_clock: int
    bounds: tuple = None


def pad_to_power_of_two(A, b, fill):
    n = len(b)
    size = 1 << int(np.ceil(np.log2(n)))
    A_pad = fill * np.eye(size)
```

```
    A_pad[:n, :n] = A
    b_pad = np.zeros(size)
    b_pad[:n] = b
    return A_pad, b_pad


def chain(n_blocks, k_neighbour=1.0, k_anchor=1.0):
    K = (2 * k_neighbour + k_anchor) * np.eye(n_blocks)
    K -= k_neighbour * (np.eye(n_blocks, k=1) + np.eye(n_blocks, k=-1))
    return K


_A3, _b3 = pad_to_power_of_two(chain(3), np.array([1.0, 0.0, 0.0]), fill=3.0)

SYSTEMS = {
    "diagonal2": LinearSystem(
        "diagonal2", "Two uncoupled springs",
        np.diag([1.0, 2.0]), np.array([1.0, 1.0]),
        n_unknowns=2, n_clock=2, bounds=(1.0, 3.0)),
    "coupled2": LinearSystem(
        "coupled2", "Two coupled blocks",
        np.array([[2.0, -1.0], [-1.0, 3.0]]), np.array([3.0, 4.0]),
        n_unknowns=2, n_clock=6),
    "chain4": LinearSystem(
        "chain4", "Chain of four blocks",
        chain(4), np.ones(4),
        n_unknowns=4, n_clock=6),
    "chain3": LinearSystem(
        "chain3", "Chain of three blocks (padded)",
        _A3, _b3,
        n_unknowns=3, n_clock=6),
}


def gershgorin_bounds(A):
    A = np.asarray(A)
    centre = np.real(np.diag(A))
    radius = np.sum(np.abs(A), axis=1) - np.abs(np.diag(A))
    return float(np.min(centre - radius)), float(np.max(centre + radius))


def evolution_time(lam_hi, n_clock):
    N = 2 ** n_clock
    return 2 * np.pi * (N - 1) / (N * lam_hi)


def clock_eigenvalues(n_clock, t):
    N = 2 ** n_clock
    k = np.arange(N, dtype=float)
    k[0] = N
    return 2 * np.pi * k / (N * t)


def rotation_angles(n_clock, t, C):
    ratio = C / clock_eigenvalues(n_clock, t)
    theta = np.zeros_like(ratio)
    ok = ratio <= 1.0 + 1e-9
    theta[ok] = 2 * np.arcsin(np.minimum(ratio[ok], 1.0))
    return theta


def parameters(system, n_clock=None):
    n_clock = system.n_clock if n_clock is None else n_clock

```

```
    lam_lo, lam_hi = system.bounds or gershgorin_bounds(system.A)
    if lam_lo <= 0:
        raise ValueError("need a positive lower bound on the eigenvalues")
    return {"n_clock": n_clock, "lam_lo": lam_lo, "lam_hi": lam_hi,
            "t": evolution_time(lam_hi, n_clock), "C": lam_lo}


def decode(joint, C, b_norm, shots=None):
    joint = np.asarray(joint, dtype=float)
    p = joint / joint.sum()
    p_success = p[1].sum()
    scale = b_norm / C
    out = {
        "p_success": p_success,
        "prob": p[1] / p_success,
        "x_abs": scale * np.sqrt(p[1]),
        "x_norm": scale * np.sqrt(p_success),
    }
    if shots is not None:
        se = np.sqrt(p[1] * (1 - p[1]) / shots)
        with np.errstate(divide="ignore", invalid="ignore"):
            out["x_abs_err"] = np.where(p[1] > 0, scale * se / (2 * np.sqrt(p[1])), 0.0)
        out["p_success_err"] = np.sqrt(p_success * (1 - p_success) / shots)
        out["accepted"] = int(round(p_success * shots))
    return out


def fidelity(rho1, x):
    x = np.asarray(x, dtype=complex)
    x = x / np.linalg.norm(x)
    return float(np.real(x.conj() @ rho1 @ x) / np.real(np.trace(rho1)))


def pauli_expectations(rho1):
    rho = rho1 / np.trace(rho1)
    return {"X": float(2 * rho[0, 1].real),
            "Y": float(2 * rho[1, 0].imag),
            "Z": float((rho[0, 0] - rho[1, 1]).real)}
```

Listing 5. `hhl_cirq.py`: the Cirq implementation, ported from the textbook listing.

```
import cirq
import numpy as np
import sympy

from hhl_common import rotation_angles


class HamiltonianSimulation(cirq.EigenGate):

    def __init__(self, _H_, t, exponent=1.0):
        cirq.EigenGate.__init__(self, exponent=exponent)
        self._H_ = np.asarray(_H_, dtype=complex)
        self.t = t
        eigen_vals, eigen_vecs = np.linalg.eigh(self._H_)
        self.eigen_components = []
        for _lambda_, vec in zip(eigen_vals, eigen_vecs.T):
            theta = -_lambda_ * t / np.pi
            _proj_ = np.outer(vec, np.conj(vec))
            self.eigen_components.append((theta, _proj_))

    def _num_qubits_(self):
```

```
        return int(np.log2(self._H_.shape[0]))

    def _with_exponent(self, exponent):
        return HamiltonianSimulation(self._H_, self.t, exponent)

    def _eigen_components(self):
        return self.eigen_components


class QFT:

    def __init__(self, qubits):
        self.qubits = list(qubits)
        n = len(self.qubits)
        self.circuit = cirq.Circuit()
        for i in range(n):
            for j in range(i):
                self.circuit.append(
                    cirq.CZ(self.qubits[j], self.qubits[i]) ** (-1.0 / 2 ** (i - j)))
            self.circuit.append(cirq.H(self.qubits[i]))
        for i in range(n // 2):
            self.circuit.append(cirq.SWAP(self.qubits[i], self.qubits[n - i - 1]))
        self.inv_circuit = cirq.inverse(self.circuit)


class ControlledUnitary(cirq.Gate):

    def __init__(self, num_qubits, num_input_qubits, U):
        self._num_qubits = num_qubits
        self.num_input_qubits = num_input_qubits
        self.num_control_qubits = num_qubits - self.num_input_qubits
        self.U = U

    def num_qubits(self):
        return self._num_qubits

    def _decompose_(self, qubits):
        qubits = list(qubits)
        input_state_qubit = qubits[:self.num_input_qubits]
        control_qubits = qubits[self.num_input_qubits:]
        for i, q in enumerate(control_qubits):
            _pow_ = 2 ** (self.num_control_qubits - i - 1)
            yield cirq.ControlledGate(self.U ** _pow_)(q, *input_state_qubit)

    def _circuit_diagram_info_(self, args):
        powers = [f"U^{2 ** (self.num_control_qubits - i - 1)}"
                  for i in range(self.num_control_qubits)]
        return ["exp(-iAt)"] * self.num_input_qubits + powers


class QuantumPhaseEstimation:

    def __init__(self, U, input_qubits, output_qubits):
        self.U = U
        self.input_qubits = list(input_qubits)
        self.output_qubits = list(output_qubits)
        self.num_input_qubits = len(self.input_qubits)
        self.num_qubits = self.num_input_qubits + len(self.output_qubits)

        self.circuit = cirq.Circuit()
        self.circuit.append(cirq.H.on_each(*self.output_qubits))
        self.circuit.append(
            ControlledUnitary(self.num_qubits, self.num_input_qubits, self.U)(
```

```
                *(self.input_qubits + self.output_qubits)))
        self.circuit += QFT(qubits=self.output_qubits).inv_circuit


class EigenValueInversion(cirq.Gate):

    def __init__(self, num_qubits, C, t):
        self._num_qubits = num_qubits
        self.C = C
        self.t = t

        self.N = 2 ** (num_qubits - 1)
        self.angles = rotation_angles(num_qubits - 1, t, C)

    def num_qubits(self):
        return self._num_qubits

    def _decompose_(self, qubits):
        base_state = self.N - 1
        for eig_val_state in range(self.N):
            if eig_val_state != 0:
                base_state = eig_val_state - 1

            qubits_to_flip = eig_val_state ^ base_state

            for q in qubits[-2::-1]:
                if qubits_to_flip % 2 == 1:
                    yield cirq.X(q)
                qubits_to_flip >>= 1

            theta = self.angles[eig_val_state]
            if theta == 0.0:
                continue

            eig_val_gate = cirq.ControlledGate(cirq.ry(theta),
                                               num_controls=self._num_qubits - 1)

            yield eig_val_gate(*qubits)

    def _circuit_diagram_info_(self, args):
        return ["k"] * (self._num_qubits - 1) + ["Ry(C/lam_k)"]


def amplitude_encoding(b, qubits):
    if np.iscomplexobj(b):
        raise ValueError("amplitude_encoding needs a real vector; prepare a complex "
                         "|b> with gates via initial_state_transforms instead")
    b = np.asarray(b, dtype=float)
    b = b / np.linalg.norm(b)
    m = len(qubits)
    ops = []
    for level in range(m):
        block = 2 ** (m - level)
        for node in range(2 ** level):
            left = b[node * block: node * block + block // 2]
            right = b[node * block + block // 2: (node + 1) * block]
            if level == m - 1:
                theta = 2 * np.arctan2(right[0], left[0])
            else:
                theta = 2 * np.arctan2(np.linalg.norm(right), np.linalg.norm(left))
            if np.isclose(theta, 0.0) or np.linalg.norm(np.r_[left, right]) == 0:
                continue
            gate = cirq.ry(theta)
```

```
            if level > 0:
                path = [(node >> (level - 1 - j)) & 1 for j in range(level)]
                gate = cirq.ControlledGate(gate, control_values=path)
            ops.append(gate(*qubits[:level + 1]))
    return ops


class HHL:

    def __init__(self, hamiltonian, b=None, initial_state_transforms=None,
                 qpe_register_size=4, C=None, t=1):
        self.hamiltonian = np.asarray(hamiltonian, dtype=complex)
        self.b = b
        self.initial_state_transforms = initial_state_transforms
        self.qpe_register_size = qpe_register_size
        self.t = t
        self.C = C
        if self.C is None:
            self.C = 2 * np.pi / (2 ** self.qpe_register_size * t)

        self.ancilla_qubit = cirq.LineQubit(0)
        self.qpe_register = [cirq.LineQubit(i)
                             for i in range(1, self.qpe_register_size + 1)]
        self.initial_state_size = int(np.log2(self.hamiltonian.shape[0]))
        self.memory = [cirq.LineQubit(i)
                       for i in range(self.qpe_register_size + 1,
                                      self.qpe_register_size + 1 + self.initial_state_size)]

    def _state_preparation(self):
        if self.initial_state_transforms is not None:
            return [op(self.memory[0]) for op in self.initial_state_transforms]
        return amplitude_encoding(self.b, self.memory)

    def _phase_estimation(self):

        self.U = HamiltonianSimulation(_H_=self.hamiltonian, t=self.t)

        return QuantumPhaseEstimation(input_qubits=self.memory,
                                      output_qubits=self.qpe_register, U=self.U)

    def build_hhl_circuit(self):
        _qpe_ = self._phase_estimation()
        self.circuit = cirq.Circuit()
        self.circuit.append(self._state_preparation())
        self.circuit += _qpe_.circuit

        _eig_val_inv_ = EigenValueInversion(num_qubits=self.qpe_register_size + 1,
                                            C=self.C, t=self.t)
        self.circuit.append(_eig_val_inv_(*(self.qpe_register + [self.ancilla_qubit])))

        self.circuit += cirq.inverse(_qpe_.circuit)
        return self.circuit

    def exact(self):
        order = [self.ancilla_qubit] + self.qpe_register + self.memory
        state = cirq.Simulator(dtype=np.complex128).simulate(
            self.build_hhl_circuit(), qubit_order=order).final_state_vector
        psi = state.reshape(2, -1, 2 ** self.initial_state_size)
        joint = np.sum(np.abs(psi) ** 2, axis=1)
        rho1 = psi[1].T @ psi[1].conj()
        return {"joint": joint, "rho1": rho1}

    def clock_distribution(self):
```

```
        circuit = cirq.Circuit(self._state_preparation()) + self._phase_estimation().circuit
        state = cirq.Simulator(dtype=np.complex128).simulate(
            circuit, qubit_order=self.qpe_register + self.memory).final_state_vector
        return np.sum(np.abs(state.reshape(2 ** self.qpe_register_size, -1)) ** 2, axis=1)

    def sample(self, shots, seed=None):
        circuit = self.build_hhl_circuit().copy()
        circuit.append(cirq.measure(self.ancilla_qubit, key='a'))
        circuit.append(cirq.measure(*self.memory, key='m'))
        result = cirq.Simulator(seed=seed).run(circuit, repetitions=shots)
        a = result.measurements['a'][:, 0]
        bits = result.measurements['m']
        index = bits @ (1 << np.arange(self.initial_state_size)[::-1])
        counts = np.zeros((2, 2 ** self.initial_state_size), dtype=int)
        np.add.at(counts, (a, index), 1)
        return counts

    def sample_pauli(self, shots, seed=None):
        circuit = self.build_hhl_circuit().copy()
        circuit.append(cirq.measure(self.ancilla_qubit, key='a'))
        circuit.append([
            cirq.PhasedXPowGate(
                exponent=sympy.Symbol('exponent'),
                phase_exponent=sympy.Symbol('phase_exponent'))(*self.memory),
            cirq.measure(*self.memory, key='m')
        ])

        params = [{'exponent': 0.5, 'phase_exponent': -0.5},
                  {'exponent': 0.5, 'phase_exponent': 0},
                  {'exponent': 0, 'phase_exponent': 0}]
        results = cirq.Simulator(seed=seed).run_sweep(circuit, params, repetitions=shots)
        out = {"accepted": {}}
        for label, result in zip(('X', 'Y', 'Z'), list(results)):

            kept = result.measurements['m'][result.measurements['a'] == 1]
            out[label] = float(1 - 2 * np.mean(kept))
            out["accepted"][label] = int(len(kept))
        return out


if __name__ == '__main__':
    A = np.array([[4.30213466 - 6.01593490e-08j, 0.23531802 + 9.34386156e-01j],
                  [0.23531882 - 9.34388383e-01j, 0.58386534 + 6.01593489e-08j]])
    t = 0.358166 * np.pi
    initial_state_transforms = [cirq.rx(1.276359), cirq.rz(1.276359)]
    _hhl_ = HHL(hamiltonian=A, initial_state_transforms=initial_state_transforms,
                qpe_register_size=4, t=t)
    print(_hhl_.build_hhl_circuit())
    print("Expected observable outputs: X = 0.144130, Y = 0.413217, Z = -0.899154")
    print("Actual:", _hhl_.sample_pauli(5000, seed=1))
```

Listing 6. `hhl_qiskit.py`: the Qiskit implementation.

```
# this file was written with the help of claude ai

import numpy as np
from qiskit import ClassicalRegister, QuantumCircuit, QuantumRegister, transpile
from qiskit.circuit.library import QFTGate, StatePreparation, UCRYGate, UnitaryGate
from qiskit.quantum_info import Statevector
from qiskit_aer import AerSimulator
from scipy.linalg import expm
```

```
from hhl_common import rotation_angles


class HHL:

    def __init__(self, A, b, n_clock=4, C=None, t=1.0):
        A = np.asarray(A, dtype=complex)
        lower = np.tril(A, -1)
        self.A = lower + lower.conj().T + np.diag(np.diag(A).real)
        self.b = np.asarray(b, dtype=complex) / np.linalg.norm(b)
        self.n_clock = n_clock
        self.t = t
        self.C = 2 * np.pi / (2 ** n_clock * t) if C is None else C

        self.m = int(np.log2(len(self.b)))
        self.memory = QuantumRegister(self.m, "b")
        self.clock = QuantumRegister(n_clock, "clock")
        self.ancilla = QuantumRegister(1, "anc")

    def _phase_estimation(self):
        qpe = QuantumCircuit(self.memory, self.clock, name="QPE")
        qpe.h(self.clock)
        for j in range(self.n_clock):
            U = UnitaryGate(expm(1j * self.A * self.t * 2 ** j), label=f"U^{2 ** j}")
            qpe.append(U.control(1), [self.clock[j], *self.memory])
        qpe.append(QFTGate(self.n_clock).inverse(), self.clock)
        return qpe

    def build_circuit(self):
        qpe = self._phase_estimation()
        qc = QuantumCircuit(self.memory, self.clock, self.ancilla)
        qc.append(StatePreparation(self.b, label="|b>"), self.memory)
        qc.append(qpe.to_gate(label="QPE"), [*self.memory, *self.clock])
        angles = rotation_angles(self.n_clock, self.t, self.C)
        rotation = UCRYGate(list(angles))
        rotation.label = "Ry(theta_k)"
        qc.append(rotation, [self.ancilla[0], *self.clock])
        qc.append(qpe.inverse().to_gate(label="QPE inverse"), [*self.memory, *self.clock])
        return qc

    def exact(self):
        state = Statevector(self.build_circuit()).data
        psi = state.reshape(2, 2 ** self.n_clock, 2 ** self.m)
        joint = np.sum(np.abs(psi) ** 2, axis=1)
        rho1 = psi[1].T @ psi[1].conj()
        return {"joint": joint, "rho1": rho1}

    def clock_distribution(self):
        qc = QuantumCircuit(self.memory, self.clock)
        qc.append(StatePreparation(self.b, label="|b>"), self.memory)
        qc.compose(self._phase_estimation(), inplace=True)
        state = Statevector(qc).data.reshape(2 ** self.n_clock, 2 ** self.m)
        return np.sum(np.abs(state) ** 2, axis=1)

    @staticmethod
    def _compile(circuit):
        return transpile(circuit, AerSimulator(), optimization_level=0, seed_transpiler=0)

    def _run(self, circuit, shots, seed):
        backend = AerSimulator(seed_simulator=seed)
        return backend.run(self._compile(circuit), shots=shots).result().get_counts()

    def _measured(self, basis="Z"):
```

```
        qc = self.build_circuit()
        if basis == "X":
            qc.h(self.memory)
        elif basis == "Y":
            qc.sdg(self.memory)
            qc.h(self.memory)
        creg = ClassicalRegister(self.m + 1, "c")
        qc.add_register(creg)
        qc.measure([*self.memory, self.ancilla[0]], creg)
        return qc

    def sample(self, shots, seed=None):
        counts = np.zeros((2, 2 ** self.m), dtype=int)
        for bitstring, n in self._run(self._measured(), shots, seed).items():
            value = int(bitstring, 2)
            counts[value >> self.m, value & (2 ** self.m - 1)] += n
        return counts

    def sample_pauli(self, shots, seed=None):
        out = {"accepted": {}}
        for offset, basis in enumerate("XYZ"):
            run_seed = None if seed is None else seed + offset
            counts = self._run(self._measured(basis), shots, run_seed)
            zeros, ones = counts.get("10", 0), counts.get("11", 0)
            out[basis] = (zeros - ones) / (zeros + ones)
            out["accepted"][basis] = zeros + ones
        return out

    def resources(self):
        compiled = transpile(self.build_circuit(), basis_gates=["cx", "u"],
                             optimization_level=1, seed_transpiler=0)
        ops = compiled.count_ops()
        return {"qubits": compiled.num_qubits, "depth": compiled.depth(),
                "cx": int(ops.get("cx", 0)), "u": int(ops.get("u", 0))}


if __name__ == "__main__":
    from hhl_common import SYSTEMS, decode, parameters

    system = SYSTEMS["coupled2"]
    p = parameters(system)
    solver = HHL(system.A, system.b, n_clock=p["n_clock"], C=p["C"], t=p["t"])
    result = decode(solver.sample(8192, seed=1), C=p["C"],
                    b_norm=np.linalg.norm(system.b), shots=8192)
    print("classical x :", np.linalg.solve(system.A, system.b))
    print("HHL |x|     :", result["x_abs"], "+-", result["x_abs_err"])
```